

Atlas Payments API

Reference, version 2.3

Published 14 January 2025. Supersedes version 2.2.

Overview

The Atlas Payments API is a JSON over HTTPS interface for creating charges, issuing refunds and reconciling settlements. All requests and responses use UTF-8. Requests with a body must send a Content-Type of application/json; other content types are rejected with a 415 status.

Base URLs

Environment	Base URL
Production	https://api.atlaspay.example/v2
Sandbox	https://sandbox.atlaspay.example/v2

The sandbox holds no real funds and is reset every Sunday at 02:00 UTC. Objects created in the sandbox are not preserved across resets and sandbox identifiers are not valid in production.

Versioning

The major version is part of the path. Breaking changes are released as a new major version and the previous major version is supported for a minimum of 18 months after its successor is published. Additive changes, such as new optional fields or new enum members, may be made within a major version without notice, so clients must ignore unrecognised fields rather than failing.

Deprecations are announced in the changelog and signalled by a Sunset response header carrying the retirement date in HTTP-date format.

Authentication

Every request carries a bearer token in the Authorization header. There is no session or cookie mechanism and no OAuth flow for server-to-server use.

Keys are environment-scoped. Production keys are prefixed `atlas_live_` and sandbox keys are prefixed `atlas_test_`. A sandbox key presented to the production base URL returns 401 with the code `key_environment_mismatch`.

Key rotation

Keys should be rotated at least every 90 days. An account may hold two active keys at once so that rotation can be performed without downtime: create the second key, deploy it, then revoke the first. Revocation takes effect within 60 seconds across all regions.

A key that has not been used for 12 months is disabled automatically and must be re-enabled from the dashboard before it will authenticate again.

Scopes

Scope	Grants
<code>charges:read</code>	Read charges and transaction listings
<code>charges:write</code>	Create charges and capture authorisations
<code>refunds:write</code>	Issue full and partial refunds
<code>settlements:read</code>	Read settlement reports and payout detail

A request whose key lacks the required scope returns 403 with the code `insufficient_scope`. The response names the missing scope in the details object.

Core endpoints

POST /charges

Creates a charge against a payment method. Returns 201 on success with the charge object in the body.

Field	Type	Required	Notes
amount	integer	yes	Minor units, e.g. 1050 for 10.50
currency	string	yes	Three-letter code, lowercase
source	string	yes	Payment method identifier
capture	boolean	no	Defaults to true
description	string	no	Maximum 500 characters
metadata	object	no	Up to 20 keys, values under 500 chars

POST /charges/{id}/refunds

Issues a refund. Omitting the amount field refunds the full remaining balance. Partial refunds may be issued repeatedly until the original charge amount is exhausted; an attempt to exceed it returns 422 with the code `refund_exceeds_charge`.

Refunds against a charge older than 180 days are rejected. In that case the funds must be returned by bank transfer outside the API.

GET /transactions

Returns a paginated listing. Pagination is cursor-based: pass the value of `next_cursor` from the previous response as the cursor query parameter. Offset pagination is not supported. The default page size is 25 and the maximum is 100.

Rate limits

The default limit is 120 requests per minute per API key, measured over a sliding 60-second window. Read endpoints and write endpoints share the same budget.

Every response carries the headers `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset`. When the limit is exceeded the API returns 429 with a `Retry-After` header in whole seconds. Clients should retry with exponential backoff and jitter rather than at a fixed interval.

Accounts processing more than 500,000 charges per month may request a raised limit of 600 requests per minute through support. The raised limit is applied per key, not per account.

Idempotency

All POST endpoints accept an `Idempotency-Key` header. The value must be unique per logical operation; a UUID version 4 is recommended. Replaying a request with the same key returns the original response, including the original status code, rather than performing the operation again.

Idempotency records are retained for 24 hours. After that window the same key may be reused and will be treated as a new operation, so clients must not rely on the key alone for long-running deduplication.

If a request with a given idempotency key is still in flight when a second request presents the same key, the second request returns 409 with the code `request_in_progress`.

Errors

Errors return a JSON body containing type, code, message and an optional details object. The message is intended for logs and should not be shown to cardholders.

Status	Code	Meaning
400	invalid_request	Malformed body or missing field
401	invalid_key	Key unknown, revoked or expired
401	key_environment_mismatch	Sandbox key used in production
403	insufficient_scope	Key lacks the required scope
404	resource_not_found	Object does not exist for this account
409	request_in_progress	Idempotency key currently in flight
422	refund_exceeds_charge	Refund total above charge amount
429	rate_limit_exceeded	Too many requests
503	processor_unavailable	Upstream processor unreachable

Webhooks

Webhook payloads are signed with HMAC SHA-256 over the raw request body, using the endpoint's signing secret. The signature and a Unix timestamp are supplied in the Atlas-Signature header. Reject any payload whose timestamp is more than five minutes old to prevent replay.

Verify the signature against the raw bytes of the body before parsing the JSON. Parsing and re-serialising first will change the byte sequence and cause verification to fail.

A delivery is considered successful on any 2xx response returned within 10 seconds. Failed deliveries are retried 5 times over 24 hours with increasing backoff, after which the event is marked undeliverable and must be replayed manually from the dashboard.